

Program 9

Organize Kubernetes Pods using Labels and Namespace

Program Structure :

----->namespace.yaml

----->pods.yaml

Procedure :

step 1 : Start minikube

minikube start

Check the status of minikube

minikube status

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program9$ minikube start
🐳 minikube v1.37.0 on Ubuntu 22.04
🌟 Using the docker driver based on existing profile
👉 Starting "minikube" primary control-plane node in "minikube" cluster
🚢 Pulling base image v0.0.48 ...
🔄 Updating the running docker "minikube" container ...
🔧 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔍 Verifying Kubernetes components...
   ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌞 Enabled addons: storage-provisioner, default-storageclass

! /usr/bin/kubectl is version 1.30.14, which may have incompatibilities with Kubernetes 1.34.0.
   ▪ Want kubectl v1.34.0? Try 'minikube kubectl -- get pods -A'
🏠 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

step 2 : Create namespace.yaml file with the following code

nano namespace.yaml

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx: ~/program9
GNU nano 6.2 namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev
---
apiVersion: v1
kind: Namespace
metadata:
  name: prod
```

step 3 : Create pods.yaml file with the following code

nano pods.yaml



GNU nano 6.2

pods.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
  namespace: dev
  labels:
    app: frontend
    env: dev
spec:
  containers:
    - name: frontend
      image: nginx
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: backend-pod
  namespace: dev
  labels:
    app: backend
    env: dev
spec:
  containers:
    - name: backend
      image: nginx
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
```

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx: ~/program9
GNU nano 6.2 pods.yaml
- containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
  namespace: prod
  labels:
    app: frontend
    env: prod
spec:
  containers:
  - name: frontend
    image: nginx
    ports:
    - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: backend-pod
  namespace: prod
  labels:
    app: backend
    env: prod
spec:
  containers:
  - name: backend
    image: nginx
    ports:
    - containerPort: 80
■
```

Step 4 : Apply the file

```
kubectl apply -f namespace.yaml
kubectl apply -f pods.yaml
```

```
verfiy the pods with namespce
kubectl get pods -n dev
kubectl get pods -n prod
```

step 5 : verfiy pods with labels

```
kubectl get pods -n dev -l app=frontend
kubectl get pods -n dev -l app=backend
kubectl get pods -n prod -l app=frontend
kubectl get pods -n prod -l app=backend
```

```

1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program9$ kubectl apply -f namespace.yaml
namespace/dev unchanged
namespace/prod unchanged
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program9$ kubectl get pods -n dev
NAME          READY   STATUS    RESTARTS   AGE
backend-pod    1/1     Running   1 (3m17s ago)  4m26s
frontend-pod   1/1     Running   1 (3m17s ago)  4m26s
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program9$ kubectl get pods -n prod
NAME          READY   STATUS    RESTARTS   AGE
backend-pod    1/1     Running   1 (4m3s ago)  5m11s
frontend-pod   1/1     Running   1 (4m3s ago)  5m11s
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program9$ kubectl delete pods --all
pod "program-8" deleted
^C1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program9$ kubectl get pods -n dev -l app=fronte
NAME          READY   STATUS    RESTARTS   AGE
frontend-pod   1/1     Running   1 (4m54s ago)  6m3s
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program9$ kubectl get pods -n dev -l app=backend
NAME          READY   STATUS    RESTARTS   AGE
backend-pod    1/1     Running   1 (5m6s ago)  6m15s
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program9$ █

```